KNOWLEDGE TRANSFER

LLM Inference & Sampling

Reverse-KT

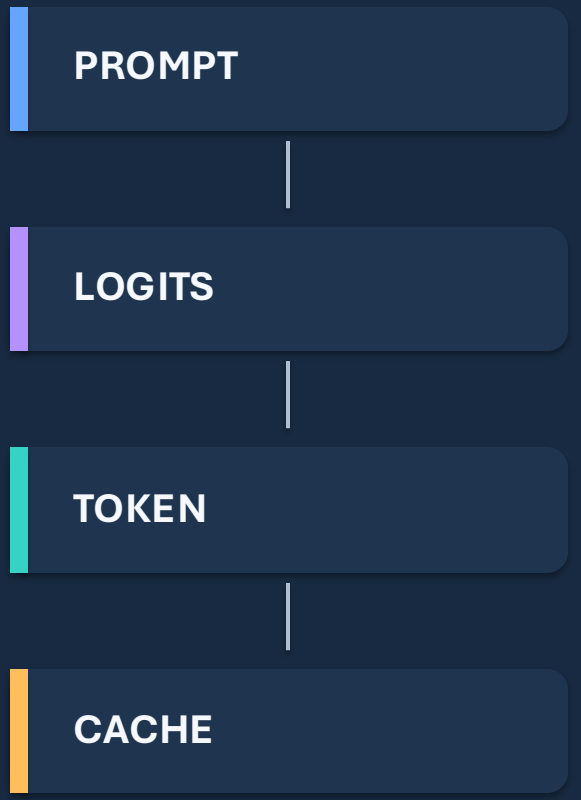
KV-cache Sampling Lab

Sampling • attention • KV cache • FastAPI • k8s • monitoring

CPU-first demo | Multi-Node | Local models | Observable behavior



Project: https://github.com/ojas-kulkarni-emergys/sampling-kv-cache-lab



Learning Tracks Covered

The KT connects course concepts to a concrete, inspectable serving system.

Hugging Face LLM course

Transformers, generation, logits, decoding and cache behavior.

Kubernetes Crash Course

Pods, Deployments, Services, persistent storage and operational commands.

DeepLearning.AI MLOps

Packaging, serving, metrics, monitoring and deployment thinking.

APPLIED IN THE LAB

A local text-generation UI, a separate inference API, a Kubernetes deployment, and measurable generation/cache behavior.

Learning resources are linked in the final handover slide.

KT objectives

What the implementation demonstrates

LLM behavior

Control token selection and inspect how generation changes with decoding settings.

Inference engineering

Separate the user interface from model execution behind a versionable API.

MLOps & operations

Containerize components, deploy repeatably, expose metrics and inspect logs.

- Understand the path from logits to token selection.
- Compare two decoding configurations under the same prompt and seed.
- Inspect measured K/V cache tensors and optional attention matrices.
- Operate the UI and inference service as separate Kubernetes workloads.

Transferrable KPIs

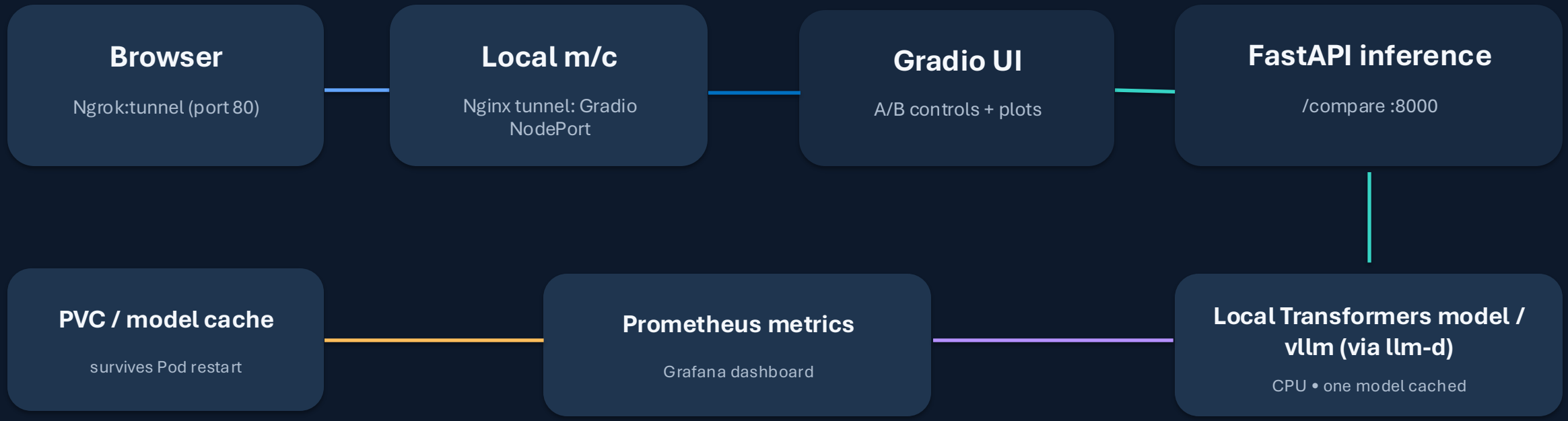
A small lab makes model-serving trade-offs visible before scaling to production.



- Serving boundary: Teams-facing experience can call a backend API instead of embedding model execution in the UI.
- Reliability: health checks, request validation, bounded generation and clear error paths.
- Operational visibility: latency, request counts, generated-token counts and cache payload metrics.
- Scale deliberately: measure CPU/RAM and concurrency before increasing replicas or enabling autoscaling.
- Scaling Deployments: llm-d with router & gateway
- IaC: Using terraform to setup infrastructure locally

Service Access

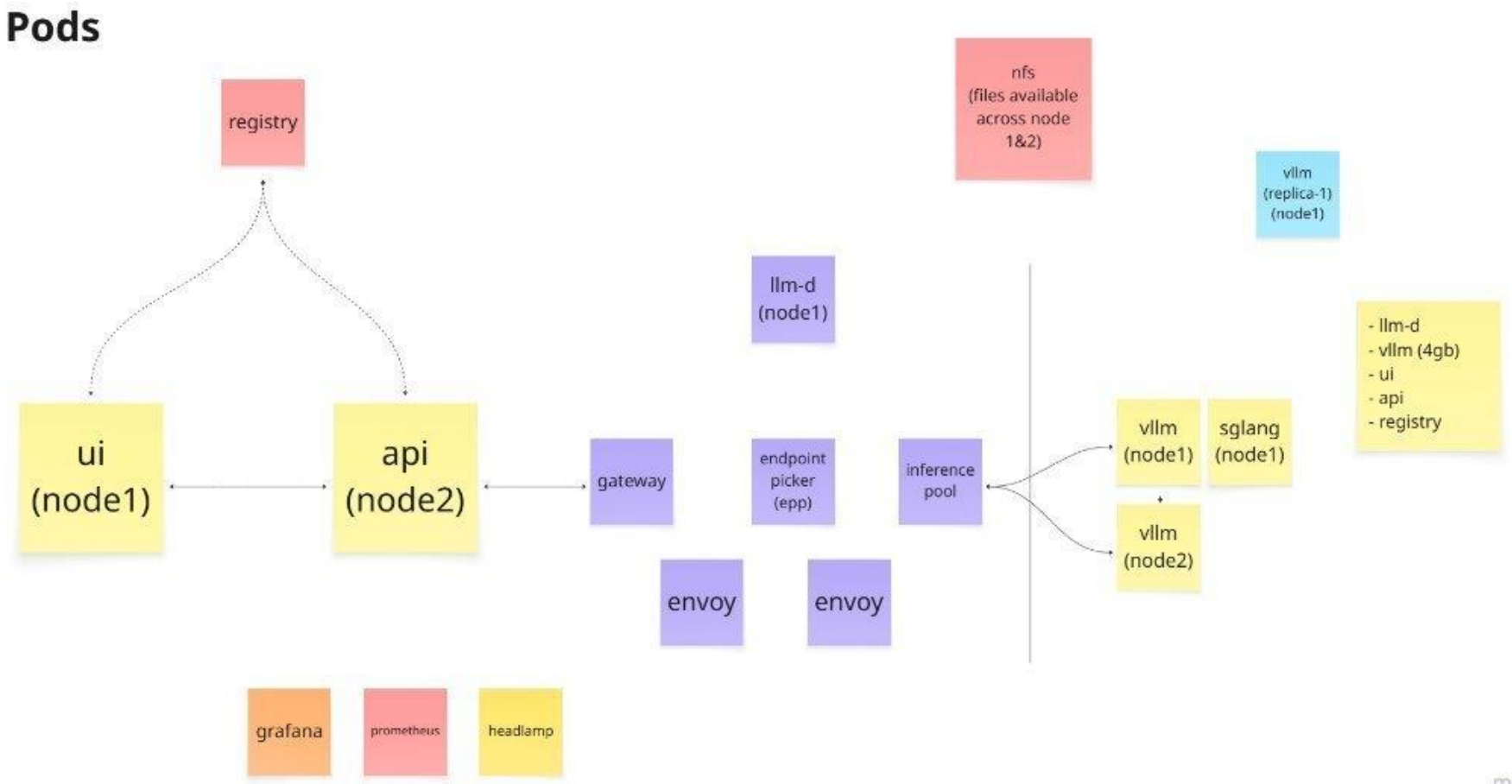
The UI and inference API can be run locally in-process or separated using INFERENCE_URL.



Kubernetes namespace: sampling-lab • Service routes UI → inference(local/llm-d) • PVC stores downloaded model files

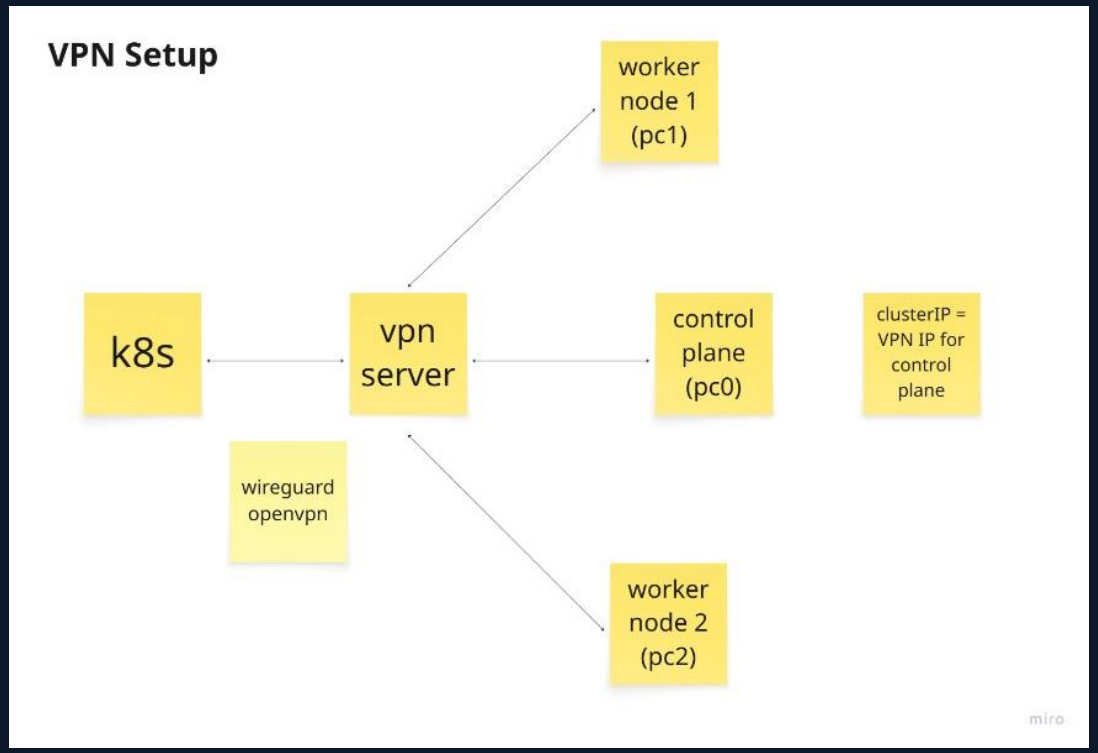
System architecture

Kubernetes k8s Pods



VPN Setup

The VPN setup uses a hosted server for the current setup, but on-prem hosting can be done with wireguard or openvpn



Terraform Scripts:

- To automate the setting up of the control plane/worker nodes
- To automate the helm installation of the setup
- To automate the setup of VPN

Project: https://github.com/ojas-kulkarni-emergys/k8s-setup

Attention: each token builds context from earlier tokens

For causal language models, a position attends to allowed earlier positions, not future tokens.

Query (Q)

What information is this token looking for?

Key (K)

What does each context position offer for matching?

Value (V)

What information is weighted and mixed into the result?

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k} + \text{causal mask})\, V$$

The demo can show a layer-0 attention heatmap: query-token position × key-token position.

Why it matters: attention explains contextual token representations; its matrices can be expensive to retain, so capture is optional in this CPU-first lab.

From logits to the next token

A/B comparison keeps the prompt fixed while changing the decoding policy.

1 LOGITS
Model scores vocabulary

2 ADJUST
Penalties / temperature

3 FILTER
Top-k, top-p, min-p, typical

4 SELECT
Sample token or expand beams

5 APPEND
Update context + cache

Greedy

Pick the highest-probability token; repeatable for the same model/context.

Sampling

Temperature and filters change which candidates can be selected.

Beam search

Track multiple candidate sequences; beam width applies to this mode.

Seed • maximum new tokens • stop string • repetition/frequency penalties are also exposed in the UI.

KV cache avoids recomputing prior token projections

The lab reports real past_key_values payloads, not a synthetic cache estimate.

Prefill

Process the prompt and build K/V tensors for its token positions.

Decode

Process each newly selected token and extend cached positions.

Inspect

See cache length, tensor shape, bytes and a K heatmap.

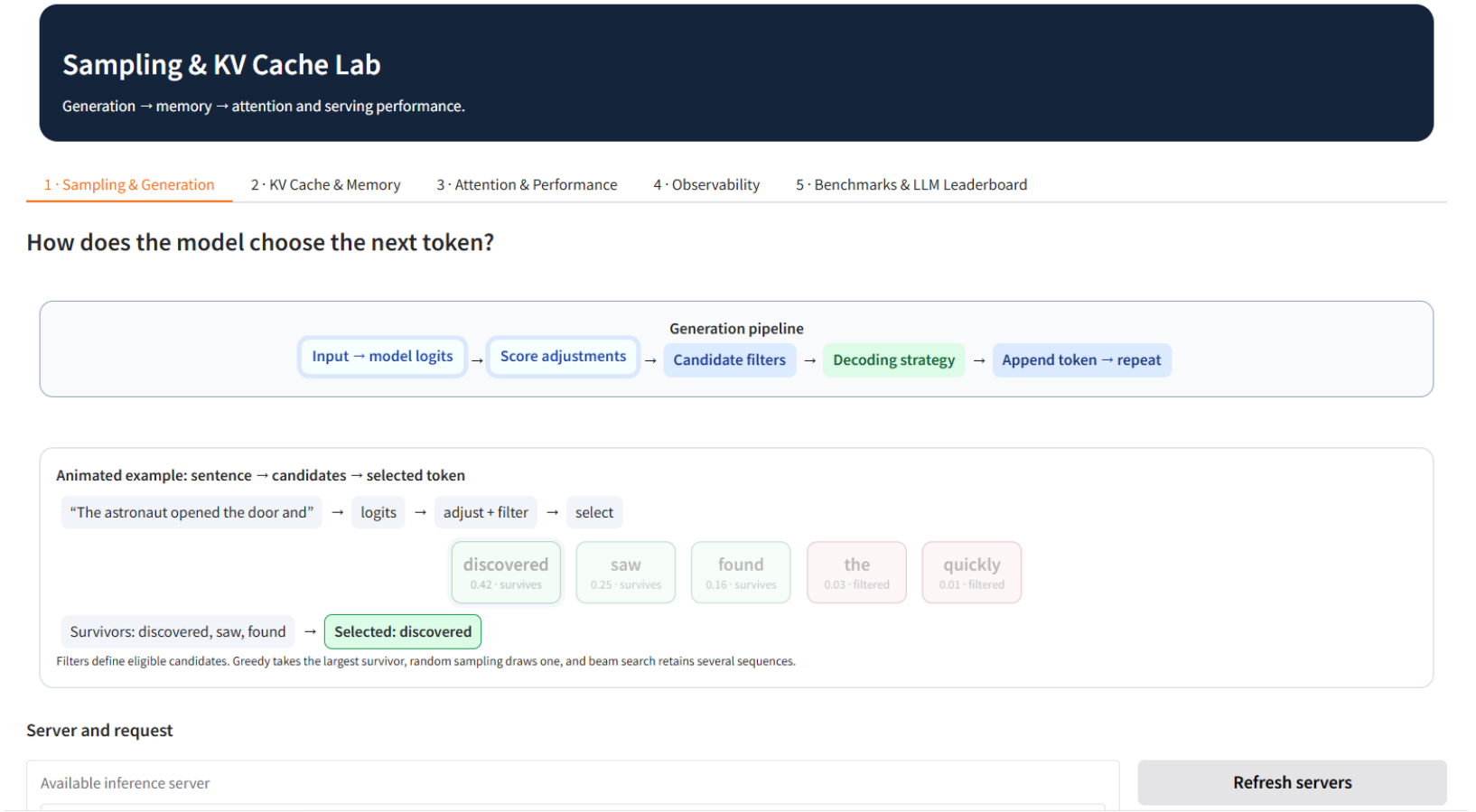
Same prompt + same processed length → same cache payload size

Different selected tokens can change later K/V values. Sampling does not change context-window capacity or bytes per cached token.

Boundary: this is a dynamic full-attention cache, not a PagedAttention or quantized-cache benchmark. Reported bytes exclude weights, allocator overhead and attention buffers.

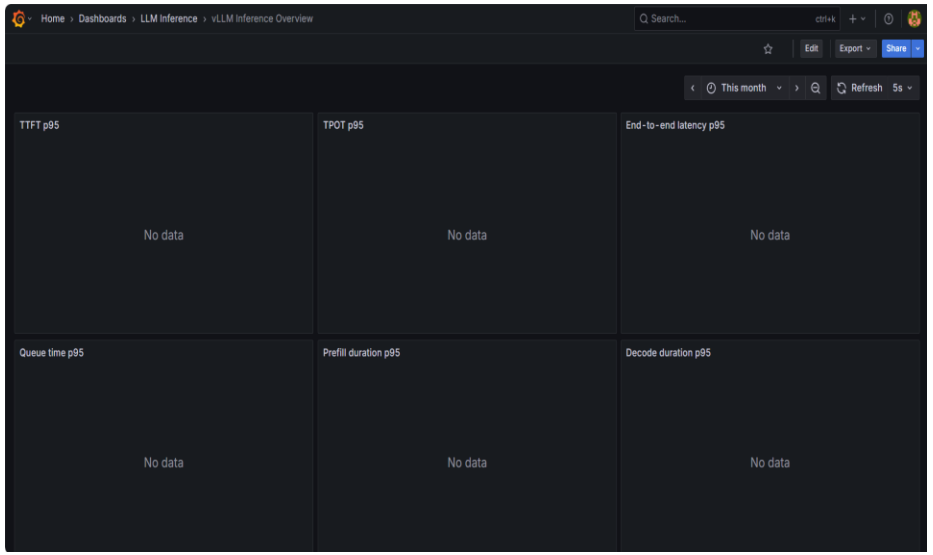
Gradio:

URL: Demo



Model & API Observability

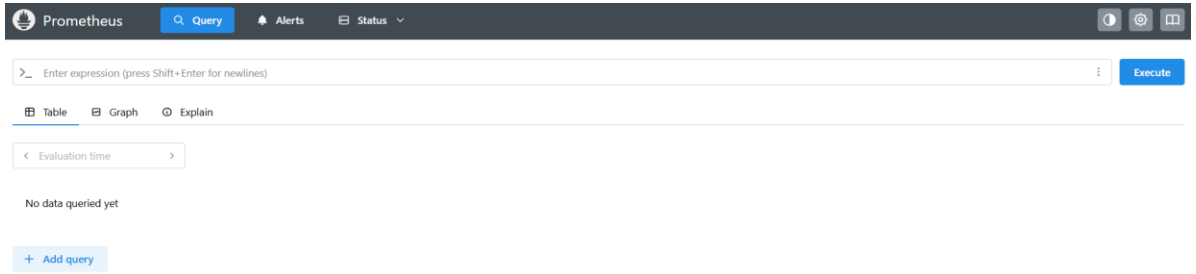
Grafana: vLLM Overview



Metrics to track:

- TTFT
- TPOT (p95)
- Tokens/sec
- KV-cache utilization
- Prefill Time (p95)
- Latency
- Throughput

Prometheus: PromQuery

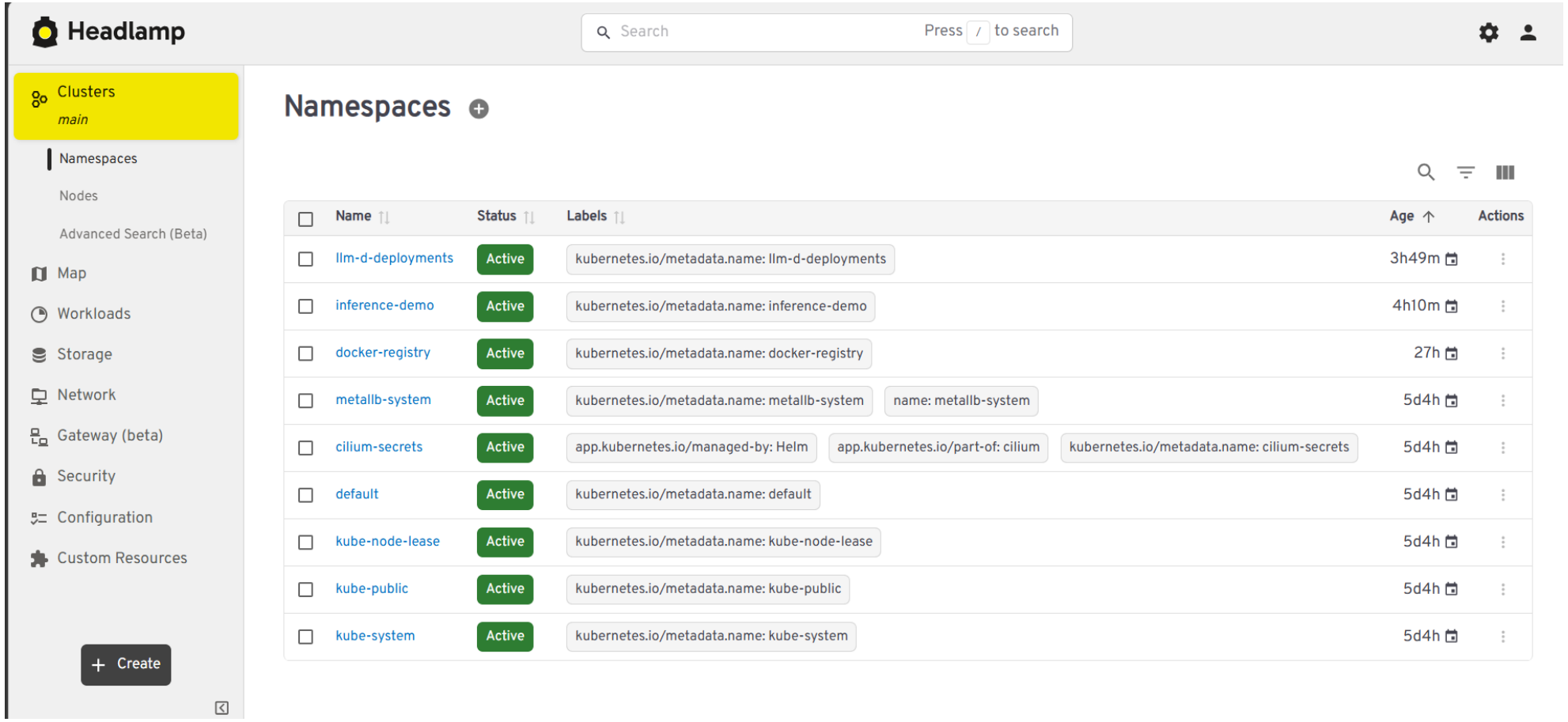


Note:

- Grafana queries Prometheus to get the metrics
- Preconfigured Dashboards are displayed on Grafana Dashboard
- Preconfigured Queries are run through PromQL

Cluster Observability

Headlamp: Dashboard







Demo Urls:

- **Gradio URL: Demo**
- **Grafana: vLLM Overview**
- **Prometheus: PromQuery**
- **Headlamp: Dashboard**

Additional Links:

- **llm-d/llm-d: State of the art inference performance with modern accelerators on Kubernetes**
- **Machines - Tailscale**
- **System Architecture | Miro Board**

Github Repo:

- Project: https://github.com/ojas-kulkarni-emergys/k8s-setup
- Project: https://github.com/ojas-kulkarni-emergys/sampling-kv-cache-lab

Thank you